

✓ SUPERVISED LEARNING

Supervised learning adalah jenis algoritma pembelajaran mesin di mana data pelatihan sudah memiliki fitur dan label yang diketahui. Metode ini melatih model untuk memahami hubungan antara fitur dan label berdasarkan data sebelumnya, sehingga model dapat memprediksi label untuk data baru yang belum diketahui.

Regresi

Regresi adalah salah satu bentuk *supervised learning* di mana model memprediksi nilai dalam bentuk angka. Contohnya:

- Jumlah es krim yang terjual dalam satu hari berdasarkan suhu, curah hujan, dan kecepatan angin.
- Harga jual sebuah properti berdasarkan luasnya (dalam meter persegi), jumlah kamar tidur, dan kondisi sosial-ekonomi di sekitarnya.
- Efisiensi bahan bakar mobil (dalam mil per galon) berdasarkan ukuran mesin, berat, lebar, tinggi, dan panjang mobil.

Pada praktikum ini kita akan mencoba menggunakan data **studi kasus prediksi penggunaan bahan bakar mobil**.

Dataset Auto-Mpg yang dapat diunduh pada <https://www.kaggle.com/datasets/uciml/autompg-dataset> merupakan data yang berkaitan dengan konsumsi bahan bakar pada kendaraan bermotor adapun variable (mil per galon). Dataset ini terdiri atas 9 kolom yaitu MPG (miles per gallon), Cylindes, Displacement, Horsepower, weight, acceleration, model year, origin dan car name. Dengan dataset ini kita ditantang untuk menentukan berapa jumlah konsumsi bahan bakar sebuah kendaraan (miles per gallon).

Berikut langkah-langkah yang harus kita lakukan untuk menjawab pertanyaan di atas.

1. Data preprocessing

Pada data processing akan dilakukan pembersihan terhadap data dan feature extraction. Adapun langkah-langkah tersebut adalah

- Langkah pertama load library yang dibutuhkan yaitu, pandas, numpy, seaborn dan matplotlib.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Setelah itu, langkah selanjutnya adalah membaca/loading dataset lalu menampilkan informasi masing-masing fitur dengan menggunakan perintah `read_csv` dan fungsi `info()`. Jangan lupa melakukan koneksi google colab ke google drive

```
from google.colab import drive
import sys
import os
drive.mount('/content/gdrive')
drivePath = ('/content/gdrive/My Drive/Comvis/Pertemuan3')
sys.path.append(drivePath)
os.chdir(drivePath)
current_dir = os.getcwd()
print(current_dir)
```

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call `drive.mount("/content/gdrive", force_remount=True)`

```
import pandas as pd

#baca dataset mpg
df = pd.read_csv('auto-mpg.csv')
df.info()
```

Berdasarkan informasi awal ini, ada hal yang aneh yaitu horsepower bertipe object. Logikanya horsepower harus bertipe float64 atau int64 karena pasti bernilai angka, namun memiliki tipe yang sama dengan car name

- Langkah selanjutnya adalah menginvestigasi nilai unik horsepower. Dengan menggunakan fungsi `unique`, ditampilkan semua isi dari horsepower untuk mengetahui apa yang terjadi.

```
df['horsepower'].unique()

array(['130', '165', '150', '140', '198', '220', '215', '225', '190',
       '170', '160', '95', '97', '85', '88', '46', '87', '90', '113',
       '200', '210', '193', '?', '100', '105', '175', '153', '180', '110',
       '72', '86', '70', '76', '65', '69', '60', '80', '54', '208', '155',
       '112', '92', '145', '137', '158', '167', '94', '107', '230', '49',
       '75', '91', '122', '67', '83', '78', '52', '61', '93', '148',
       '129', '96', '71', '98', '115', '53', '81', '79', '120', '152',
```

```
'102', '108', '68', '58', '149', '89', '63', '48', '66', '139',
'103', '125', '133', '138', '135', '142', '77', '62', '132', '84',
'64', '74', '116', '82'], dtype=object)
```

Ternyata ada horsepower yang bernilai "?". Hal ini dapat diartikan sebagai MISSING VALUE. Berikut ini adalah data-data yang bernilai "?"

```
df[df['horsepower']=='?']
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
32	25.0	4	98.0	?	2046	19.0	71	1	ford pinto
126	21.0	6	200.0	?	2875	17.0	74	1	ford maverick
330	40.9	4	85.0	?	1835	17.3	80	2	renault lecar deluxe
336	23.6	4	140.0	?	2905	14.3	80	1	ford mustang cobra
354	34.5	4	100.0	?	2320	15.8	81	2	renault 18i
374	23.0	4	151.0	?	3035	20.5	82	1	amc concord dl

Karena jumlah missing value sedikit, maka data ini akan diabaikan. Jadi record yang berisi data kosong ini dihapus saja, sehingga data yang awalnya 398 berkurang menjadi 392.

```
#menghilangkan semua missing value dan karakter "?"
df = df.dropna()
df = df[df['horsepower'] != '?']
#memperbaiki tipe data horsepower
df['horsepower'] = pd.to_numeric(df['horsepower'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   mpg              392 non-null    float64
1   cylinders        392 non-null    int64
2   displacement     392 non-null    float64
3   horsepower       392 non-null    int64
4   weight           392 non-null    int64
5   acceleration     392 non-null    float64
6   model year      392 non-null    int64
7   origin           392 non-null    int64
8   car name        392 non-null    object
dtypes: float64(3), int64(5), object(1)
memory usage: 30.6+ KB
```

Selanjutnya, dilakukan Seleksi Fitur. Oleh karena itu akan dilihat apa saja isi dari dataset dan bagaimana korelasinya menggunakan pairplot

```
sns.pairplot(df)
plt.show()
```



Atau anda bisa menghitung nilai korelasi antar variable menggunakan Pearson Correlation. Berikut adalah nilai interpretasi dari Pearson Correlation

Absolute Magnitude of the Observed Correlation Coefficient

0.00–0.10
0.10–0.39
0.40–0.69
0.70–0.89
0.90–1.00

Interpretation

Negligible correlation
Weak correlation
Moderate correlation
Strong correlation
Very strong correlation

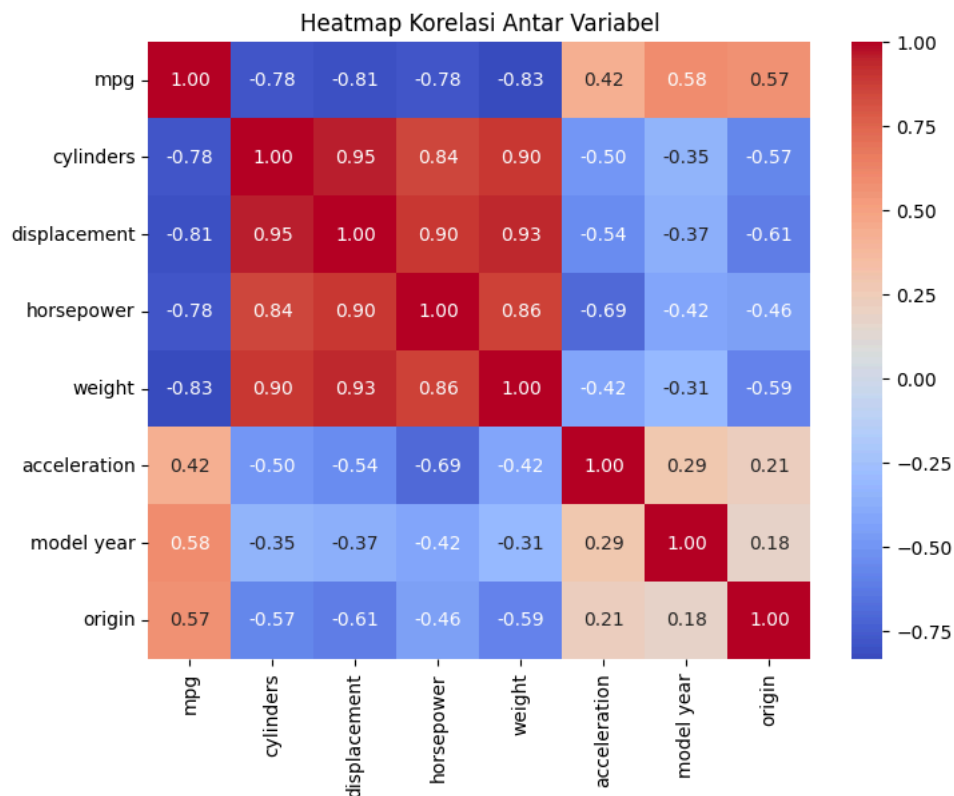
Several stratifications (with different cutoff points) have been previously published.

(sumber: https://journals.lww.com/anesthesia-analgesia/fulltext/2018/05000/correlation_coefficients_appropriate_use_and.50.aspx).

```
# Hapus kolom non-numerik (misalnya, kolom 'name' jika ada)
df = df.select_dtypes(include=['number'])
df.info()

# Visualisasi heatmap korelasi
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Heatmap Korelasi Antar Variabel")
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0    mpg         392 non-null    float64
1   cylinders   392 non-null    int64
2  displacement 392 non-null    float64
3   horsepower  392 non-null    int64
4    weight     392 non-null    int64
5   acceleration 392 non-null    float64
6   model year  392 non-null    int64
7    origin     392 non-null    int64
dtypes: float64(3), int64(5)
memory usage: 27.6 KB
```



Berdasarkan grafik dan heatmap di atas, kolom model year, origin, dan acceleration memiliki nilai korelasi yang tidak terlalu kuat, jadi ketiga kolom ini tidak akan kita gunakan untuk analisis selanjutnya.

Oleh karena ini dipilihlah 4 Fitur yang akan digunakan yaitu cylinders, displacement, horsepower dan weight.

```
df_features = df[['cylinders', 'displacement', 'horsepower', 'weight']]
df_features = df_features.apply(pd.to_numeric)
df_labels = df['mpg']
```

Karena di Indonesia lebih familiar dengan Kilogram dibandingkan pound (LBS) maka dilakukan konversi terhadap data menjadi kilogram dan miles per gallon menjadi KM per liter. 1 LBS setara dengan 0.4535923 Kg, Miles setara dengan 1.60934 KG dan 1 gallon setara dengan 3.78541 Liter. Untuk konversi LBS ke liter tinggal mengalikan dengan 0.4535923 dan Miles per Gallon ke KM/L mengalikan dengan 1.60934/3.78541

```
#transformasi data/ standardisasi data
#di Indonesia kita menggunakan satuan KG
df_features['weight'] = df_features['weight'] * 0.4535923
#transformasi M/G menjadi KM/L
df_labels = df_labels * 0.4251437
```

Selanjutnya adalah normalisasi data agar ukuran data seragam, perlu diingat bahwa metode ini sangat sensitive dengan outlier jadi normalisasi adalah salah satu cara untuk mengurangi resiko tersebut.

```
df_features = (df_features - df_features.mean()) / df_features.std()
```

Langkah terakhir pada fase ini adalah pembagian data menjadi data latih dan data uji dengan porsi 80% dan 20%

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df_features, df_labels, test_size=0.2)
```

2. Pelatihan Model

Model akan dilatih menggunakan Algoritma Regresi Linear dari Scikit- Learn.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

```
▼ LinearRegression ⓘ ?
LinearRegression()
```

3. Evaluasi Model

Model akan dievaluasi menggunakan mean square error dan R score Regresi Logistik:

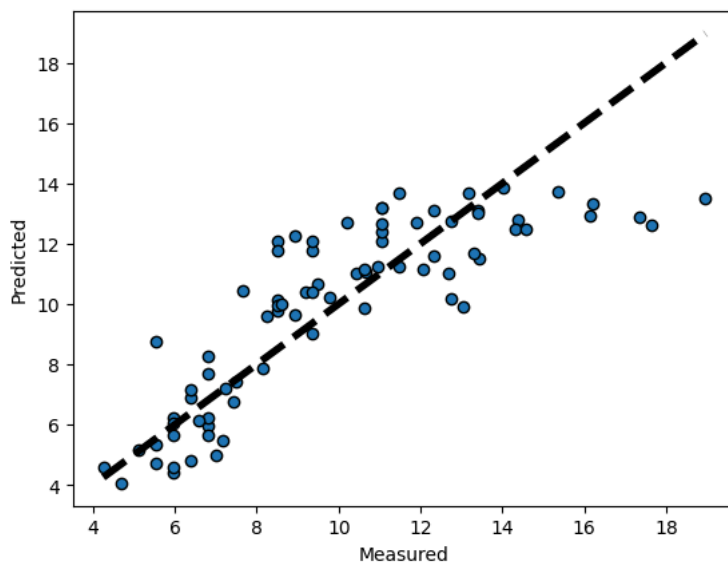
```
from sklearn.metrics import mean_squared_error, r2_score

y_pred = model.predict(X_test)

print("Mean Squared Error: %.2f" % mean_squared_error(y_test, y_pred))
print("R2: %.2f" % r2_score(y_test, y_pred))
```

```
Mean Squared Error: 3.99
R2: 0.63
```

```
#plot perbandingan prediksi dengan nilai asli
fig, ax = plt.subplots()
ax.scatter(y_test, y_pred, edgecolors=(0, 0, 0))
ax.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=4)
ax.set_xlabel('Measured')
ax.set_ylabel('Predicted')
plt.show()
```



KLASIFIKASI

Klasifikasi adalah salah satu bentuk supervised learning di mana label mewakili kategori atau kelas tertentu. Ada dua jenis klasifikasi yang umum digunakan:

1. Binary Classification

Binary classification menentukan apakah suatu objek termasuk dalam satu kelas tertentu atau tidak. Dengan kata lain, model ini memprediksi salah satu dari dua kemungkinan hasil yang bersifat saling eksklusif (misalnya ya/tidak, positif/negatif). Contohnya:

- Apakah seorang pasien berisiko terkena diabetes berdasarkan berat badan, usia, kadar gula darah, dan faktor lainnya?
- Apakah seorang nasabah akan gagal membayar pinjaman berdasarkan pendapatan, riwayat kredit, dan usia?
- Apakah pelanggan dalam daftar email akan menanggapi penawaran pemasaran berdasarkan data demografi dan riwayat pembelian?

Dalam setiap kasus, model memberikan jawaban ya/tidak atau positif/negatif untuk satu kategori tertentu.

2. Multiclass classification

Multiclass classification memperluas konsep klasifikasi biner dengan memprediksi satu dari beberapa kategori yang mungkin.

Contohnya:

- Menentukan spesies penguin berdasarkan ukuran tubuhnya (Adelie, Gentoo, atau Chinstrap).
- Menentukan genre film berdasarkan pemeran, sutradara, dan anggaran (komedi, horor, romansa, petualangan, atau fiksi ilmiah).

Dalam klasifikasi ini, setiap objek hanya dapat termasuk dalam satu kelas (misalnya, seekor penguin tidak bisa sekaligus menjadi Gentoo dan Adelie).

Namun, ada juga klasifikasi multilabel, di mana satu objek bisa memiliki lebih dari satu label. Misalnya, sebuah film bisa memiliki lebih dari satu genre, seperti fiksi ilmiah dan komedi sekaligus.

Pada praktikum ini kita akan mencoba menggunakan data **studi kasus pendeteksian penyakit diabetes**.

Dataset dalam studi kasus ini adalah dataset PIMA Indians diabetes yang dapat diunduh pada tautan

<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>.

Data ini diperoleh dari UCI yang aslinya dimiliki oleh National Institute of Diabetes and Digestive and Kidney Diseases sebagai dataset yang diberikan secara bebas kepada siapa saja yang ingin melakukan penelitian. Dataset Pima ini terdiri dari 768 data klinis yang semuanya berasal dari jenis kelamin wanita dengan umur sekurang - kurangnya 21 tahun. Data ini memiliki 8 atribut dengan target output positif diabetes (ditunjukkan dengan output = 1) dan negatif diabetes (ditunjukkan dengan output = 0). Daftar atribut data diabetes ditunjukkan pada Tabel di bawah ini

Keterangan	Satuan	Nama Atribut	Tipe data
Jumlah kehamilan	-	jHamil	Numerik
Kadar glukosa	mg/dl	jGlukosa	Numerik
Tekanan darah diastolic	mm Hg	tekananDarah	Numerik
Ketebalan kulit pada trisep	Mm	ketebalanKulit	Numerik
Serum insulin	mu U/ml	serumInsulin	Numerik
Massa indeks tubuh	kg/ m ²	IMB	Numerik
Fungsi riwayat diabetes	-	RiyawatDiabetes	Numerik
Umur	Tahun	Umur	Numerik
Positif diabetes	Positif Diabetes(1) Negatif Diabetes(0)	Class / Label	Nominal

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('diabetes.csv')
df.head(10)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
5	5	116	74	0	0	25.6	0.201	30	0
6	3	78	50	32	88	31.0	0.248	26	1
7	10	115	0	0	0	35.3	0.134	29	0
8	2	197	70	45	543	30.5	0.158	53	1
9	8	125	96	0	0	0.0	0.232	54	1

1. Preprocessing

Sebelum dilakukan proses klasifikasi, data harus dipersiapkan terlebih dahulu agar siap untuk diolah (dikenal dengan istilah pre-processing) dengan tujuan meminimalkan kesalahan dan mengoptimalkan model yang akan dihasilkan. Adapun tahap-tahap persiapan yang dilakukan adalah:

Penangan data yang tidak lengkap Berdasarkan info dataset, tidak terdapat data yang hilang. Semua kolom memiliki jumlah item yang sama

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            768 non-null    int64
1   Glucose                768 non-null    int64
2   BloodPressure          768 non-null    int64
3   SkinThickness          768 non-null    int64
4   Insulin                768 non-null    int64
5   BMI                   768 non-null    float64
6   DiabetesPedigreeFunction 768 non-null    float64
7   Age                   768 non-null    int64
8   Outcome               768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

Namun jika diperhatikan ada nilai yang aneh pada skintickness dan insulin dimana ada yang bernilai 0. Nilai nol tersebut tidak mungkin karena insulin pasti nilainya diatas 0. Setelah ditelusuri, ada 5 kolom yang bernilai 0 diantaranya

'Glucose','BloodPressure','SkinThickness','Insulin', dan 'BMI', yang mana kolom ini tidak mungkin memiliki nilai 0. Langkah selanjutnya adalah mengganti nilai 0 sebagai nilai yang hilang.

```
df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']] = df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']].replace(0, np.nan)
```

	0
Pregnancies	0
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	0

```
dtype: int64
```

Dari hasil analisa dataset Pima Indians dapat diketahui bahwa tidak semua atribut memiliki nilai yang lengkap, dimana kelengkapan atribut ini akan menentukan seberapa baik hasil dari pengklasifikasi. Jumlah data tidak lengkap pada masing-masing atribut yaitu atribut jumlah glukosa sebanyak 5, atribut tekanan darah sebanyak 35, atribut ketebalan trisep sebanyak 227, atribut insulin sebanyak 374, atribut IMB sebanyak 11, sedangkan atribut umur dan kelas memiliki nilai yang lengkap. Atribut yang memiliki missing value akan dilakukan proses imputation menggunakan pendekatan median

```
df['Glucose'] = df['Glucose'].fillna(df['Glucose'].median())
df['BloodPressure'] = df['BloodPressure'].fillna(df['BloodPressure'].median())
df['SkinThickness'] = df['SkinThickness'].fillna(df['SkinThickness'].median())
df['Insulin'] = df['Insulin'].fillna(df['Insulin'].median())
df['BMI'] = df['BMI'].fillna(df['BMI'].median())

df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

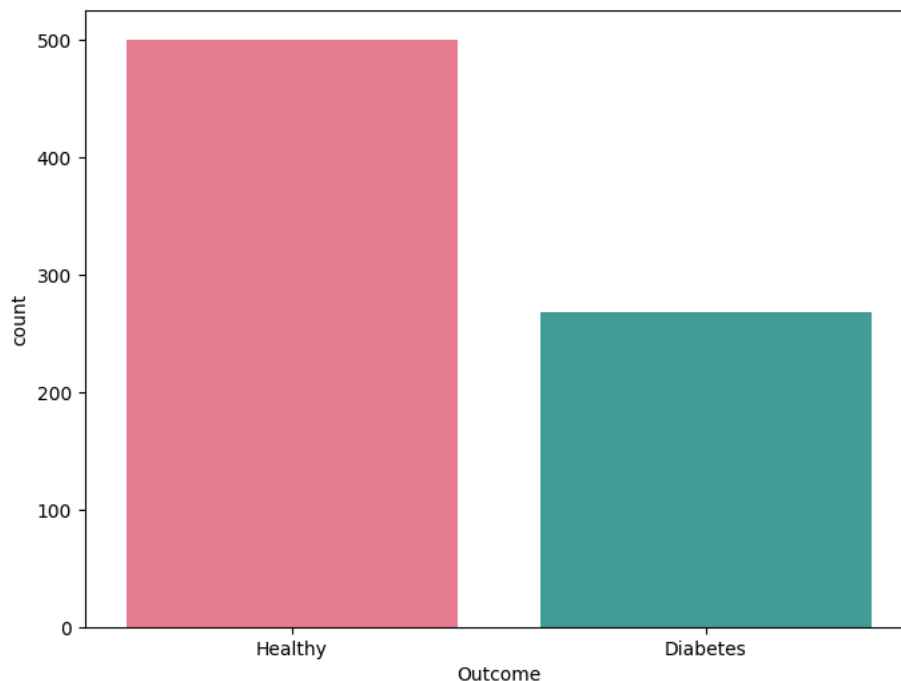
dtype: int64

```
from matplotlib.pyplot import figure, show

plt.figure(figsize=(8,6))
ax = sns.countplot(x=df['Outcome'], hue="Outcome", data=df, palette="husl", legend=False)
ax.set_xticks([0, 1])
ax.set_xticklabels(["Healthy", "Diabetes"])
healthy, diabetes = df["Outcome"].value_counts().values
print("Healthy: ", healthy)
print("Diabetes: ", diabetes)

# Tampilkan plot
plt.show()
```

Healthy: 500
Diabetes: 268



2. Hasil Pelatihan dan Pengujian

```
from sklearn.model_selection import train_test_split

X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

logmodel = LogisticRegression()
logmodel.fit(X_train, y_train)
predictions = logmodel.predict(X_test)
```

```
print("confusion matrix:\n", confusion_matrix(y_test, predictions))
print("classification report:\n", classification_report(y_test, predictions))
print("accuracy score:\n%.2f" % accuracy_score(y_test, predictions))
```

```
confusion matrix:
[[82 17]
 [21 34]]
classification report:
              precision    recall  f1-score   support

     0       0.80      0.83      0.81         99
     1       0.67      0.62      0.64         55

 accuracy          0.75
 macro avg       0.73      0.72      0.73
weighted avg       0.75      0.75      0.75

accuracy score:
0.75
```

◆ Gemini

```
!jupyter nbconvert --to html [ComVis]_Pertemuan_3_Supervised_Learning.ipynb
```

```
[NbConvertApp] WARNING | pattern '[ComVis]_Pertemuan_3_Supervised_Learning.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.
```

```
WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.
```

Options

```
=====
```

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

```
<cmd> --help-all
```

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

generate default config file

Equivalent to: [--JupyterApp.generate_config=True]

-y

Answer yes to any questions instead of prompting.

Equivalent to: [--JupyterApp.answer_yes=True]

--execute

Execute the notebook prior to export.

Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors

Continue notebook execution even if one of the cells throws an error and include the error message in the cell output

Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin

read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'

Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout

Write notebook output to stdout instead of files.

```
Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
  Run nbconvert in place, overwriting the existing notebook (only
    relevant when converting to notebook format)
  Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directories=
--clear-output
  Clear output of current file and save in place,
    overwriting the existing notebook.
  Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directories=
--clear-output
```